

2402 fzu OOP Lancer 课程设计

YAML front matter generator of Markdown

流程

这是一份初学者的课程设计，简单的实现了一个这样的流程：

1. 识别工作目录下的（默认为draft.md） markdown格式的草稿
2. 识别该草稿的title和date（一个基本的使用github pages托管的博客博客框架如Jekyll，Hexo等，其生成的draft的yaml前页所包含的项）
3. 在询问后设置文章的标题，文件名，生成对应的资源文件夹，设置日期，时间，标签，摘要等
4. 按照上述所选项更新yaml前页
5. 在工作目录的父目录下寻找名为_post的子目录，将生成后的文章保存在里面

功能

并实现了如下功能：

1.全自动流程，简明询问，标准化输入（对定项输入采取输入检查 `std::cin.fail()`，非定项输入采取无害化处理 `buffer`）

```
bool vaildinput_yon(){
    char p;
    do {
        cin >> p;
        p = toupper(p);
        cin.ignore(numeric_limits<streamsize>::max(), '\n');
        if (cin.fail()) {
            cin.clear();
            cerr << "input stream error!" << endl;
        } else if (p != 'Y' && p != 'N') cerr << "invalid choice, please enter y or n." << endl;
    } while (p != 'Y' && p != 'N');
    return p == 'Y';
}
```

功能

2.自动识别打开文件时的日期与时间 <windows.h>, <chrono>

```
DefaultData::bool initTime() {  
    auto now = chrono::system_clock::now();  
    time_t now_time_t = chrono::system_clock::to_time_t(now);  
  
    tm* now_tm = localtime(&now_time_t);  
    if (now_tm == nullptr) return false; //  
  
    ostringstream oss;  
    oss << put_time(localtime(&now_time_t), "%Y/%m/%d %H:%M:%S");  
    string formattedDateTime = oss.str();  
  
    sysdate = formattedDateTime.substr(0, 10);  
    systime = formattedDateTime.substr(11);  
    return true;  
}
```

功能

3.有标签历史使用列表（存放在运行目录下的config.json中） `json.hpp`

```
{  
    "tags": [  
        "test1",  
        "test2"  
    ]  
}
```

```
#include "json.hpp"  
using json = nlohmann::json;  
json data = json::parse(file);  
articleTagslist.push_back(data["tags"][i - 1].get<string>());  
...
```

功能

4.可简单识别markdown正文中的代码块并将代码种类填入标签中 `<regex>`

```
YAMLProcessor::void parseCodeBlocks() {  
    regex codeBlockRegex("` `[a-zA-Z+]+");  
    smatch match;  
    string temp = bodyContent;  
    while (regex_search(temp, match, codeBlockRegex)) {  
        string lang = match[1].str();  
        transform(lang.begin(), lang.end(), lang.begin(), ::toupper);  
        if (lang == "CPP") lang = "C++";  
        if (lang != "IN" && lang != "OUT" && lang != "ANS" && // 纯文本  
            find(tags.begin(), tags.end(), lang) == tags.end()) tags.push_back(lang);  
        temp = match.suffix().str();  
    }  
}
```

功能

5.可调用deepseek的api来自动对文本生成摘要（需手动在运行目录下的config.json中配置api key）

```
string loadapi() {  
    try {  
        ifstream config_file("config.json");  
        if (!config_file.is_open()) {  
            cerr << "error: can't open config.json" << endl;  
            return "";  
        }  
        json config = json::parse(config_file);  
        return config["api"]["deepseek"]["key"].get<string>();  
    } catch (const exception& e) {  
        cerr << "configuration read error: " << e.what() << endl;  
        return "";  
    }  
}
```

功能

```
string call_deepseek_api(const string& input) {
    const string api_key = loadapi();
    const string endpoint = "/v1/chat/completions";
    const string host = "api.deepseek.com";
    httplib::SSLClient client(host, 443); // https
    // 设置请求prompt
    string prompt = "default\n" + input;
    // 构造请求JSON
    json request_body = {
        {"model", "deepseek-chat"},
        {"messages", {
            {
                {"role", "user"},
                {"content", prompt}
            }
        }
    },
    {"temperature", 0.7},
    {"max_tokens", 2048}
};
    // 设置请求头
    httplib::Headers headers = { {"Content-Type", "application/json"}, {"Authorization", "Bearer " + api_key} };
    // 发送POST请求
    auto res = client.Post(
        endpoint.c_str(),
        headers,
        request_body.dump(),
        "application/json"
    );
    // 处理响应
    if (res && res->status == 200) {
        try {
            json response_json = json::parse(res->body);
            return response_json["choices"][0]["message"]["content"].get<string>();
        } catch (const std::exception& e) {
            cerr << "JSON parsing error: " << e.what() << endl;
            return "";
        }
    } else {
        if (res) cerr << "API request failed with status code: " << res->status << ", echo: " << res->body << endl;
        else cerr << "work request failed" << endl;
        return "";
    }
}
```


优点

1. 进行了较良好的封装，将默认选项与配置文件部分 `DefaultData` 与markdown文件类 `YAMLProcessor` 的逻辑处理解耦，较符合面向对象程序设计的思路与初衷
2. 使用了较多的 `try/catch/throw` 与 `std::cerr`，对可预料的错误都进行了可视性报错并尽量弥补，代码的鲁棒性较高
3. 使用了较标准的工程流程、清晰的注释与权限控制，使用有意义且规范的命名，代码的规范性与可读性较高，便于和别人进行协作与对接

```
/**作用  
 * @note 注意事项  
 * @param x 参数  
 * @return 返回值  
 */
```

4. 内存计算较为清楚，所有变量的申请与文件的读入都在使用周期结束时做了释放 `delete` 与关闭 `file.close()`，内存管理较为安全高效

缺点

作为一份初学者的课程设计，有如下缺点是在有限的开发时间内在所难免的：

1.两个类间的关系（继承）并未在一开始厘清，导致代码的逻辑略有混乱，虽做了封装且能稳定运行，但内部逻辑需要重构

黑箱->分割->通信 注重整体结构 注重对象间联系 封装接口 清晰意图

2.代码长度较长（700line+），暂时还未使用头文件和命名空间进行分割，影响了代码的模块化程度，不利于后续的扩展与维护。

缺点

3.部分期望实现的功能并未在工期内实现，如：

1. 使用正则表达式匹配完成对markdown中一级标题的识别与安全化处理（github pages的通常流程中，一级标题不该出现在正文中。期望达到的效果：如识别到一个一级标题则询问是否作为本文title，识别到多个一级标题则自动下调所有标题等级）
2. 对置顶等级的持久化设置（yaml前页给定的参数是数字越大则置顶等级越高，期望达到的效果为：和标签历史使用列表一样，对所有置顶过的文章名称存储在列表中，自动按照置顶关系分配历史文件与现在文件的置顶参数）
3. 还未学习Qt，未做图形化界面或插件挂载，事实上本项目最开始设想：作为插件挂载在Typora上，监听文件更新后自动实现上述流程，然而因还未做学习而搁置

展望

1. 重构代码逻辑，用更符合现代面向对象编程的逻辑完成本项目，学习并使用更多便捷的新特性
2. 后续实现上述所期望的功能
3. 学习使用CMake工具链，自动化编译流程
4. （泛化）实现一个文件操作接口，帮助配置一些非 workflow 内容的文件如：静态HTML中的CSS样式表等

一些想说的话

~~vibe-codeing~~ 知其然而知其所以然

~~网课 + 脑测~~ 啃书 + 实践

Olwiki Cpppp

闭门造车 -> 出门合辙

Personal Blog -> Github, Stack Overflow

善用工具(好的文本编辑器, 思维导图, AI等) 统筹整体 知行合一

谢谢大家

2025/04/27 王智壹